Assignment 5: Deterministic and Randomized Quick Sort

Program

```
#include <algorithm>
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <vector>
using namespace std;

int deterministicPartition(vector<int>& arr, int low, int high) {
    int pivot = arr[low];
    int left = low + 1, right = high;
    while (true) {
        while (left <= right && arr[left] <= pivot) left++;
        while (left <= right && arr[right] >= pivot) right--;
        if (left > right) break;
        swap(arr[left], arr[right]);
    }
    swap(arr[low], arr[right]);
    return right;
}

void deterministicQuickSort(vector<int>& arr, int low, int high,
                            int& comparisons, int& swaps) {
    if (low < high) {
        int pivotIndex = deterministicPartition(arr, low, high);
        comparisons += high - low;
        swaps += 2;
        deterministicQuickSort(arr, low, pivotIndex - 1, comparisons, swaps);
        deterministicQuickSort(arr, pivotIndex + 1, high, comparisons, swaps);
    }
}

int randomizedPartition(vector<int>& arr, int low, int high) {
    int pivotIndex = low + rand() % (high - low + 1);
    swap(arr[low], arr[pivotIndex]);
    int pivot = arr[low];
    int left = low + 1, right = high;
    while (true) {
        while (left <= right && arr[left] <= pivot) left++;
        while (left <= right && arr[right] >= pivot) right--;
        if (left > right) break;
        swap(arr[left], arr[right]);
    }
    swap(arr[low], arr[right]);
    return right;
}

void randomizedQuickSort(vector<int>& arr, int low, int high,
```

```
                        int& comparisons, int& swaps) {
    if (low < high) {
        int pivotIndex = randomizedPartition(arr, low, high);
        comparisons += high - low;
        swaps += 2;
        randomizedQuickSort(arr, low, pivotIndex - 1, comparisons, swaps);
        randomizedQuickSort(arr, pivotIndex + 1, high, comparisons, swaps);
    }
}

void printArray(const vector<int>& arr) {
    for (int num : arr) cout << num << " ";
    cout << endl;
}

int main() {
    srand(time(0));
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    vector<int> arr1(n);
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) cin >> arr1[i];
    vector<int> arr2 = arr1;
    int ch;
    while (1) {
        int comparisonsDet = 0, swapsDet = 0;
        int comparisonsRand = 0, swapsRand = 0;
        cout << "1.Deterministic Quick Sort\n";
        cout << "2.Randomized Quick Sort\n";
        cout << "3.Exit\n";
        cout << "Enter your choice: ";
        cin >> ch;
        switch (ch) {
            case 1:
                cout << "Original array for deterministic quick sort:\n";
                printArray(arr1);
                deterministicQuickSort(arr1, 0, arr1.size() - 1,
                                       comparisonsDet, swapsDet);
                cout << "Sorted array (Deterministic):\n";
                printArray(arr1);
                cout << "Deterministic Quick Sort Comparisons: "
                     << comparisonsDet << endl;
                cout << "Deterministic Quick Sort Swaps: " << swapsDet << endl;
                break;
            case 2:
                cout << "\nOriginal array for randomized quick sort:\n";
                printArray(arr2);
                randomizedQuickSort(arr2, 0, arr2.size() - 1,
                                    comparisonsRand, swapsRand);
                cout << "Sorted array (Randomized):\n";
                printArray(arr2);
```

```
                cout << "Randomized Quick Sort Comparisons: "
                     << comparisonsRand << endl;
                cout << "Randomized Quick Sort Swaps: " << swapsRand << endl;
                break;
            case 3:
                return 0;
        }
    }
}
```

Time Complexity (TC)

Average: O(n log n); Worst case: O(n^2) for both variants.

Space Complexity (SC)

O(log n) average recursion stack; O(n) worst case.

Input

```
5
5 3 8 1 2
```

Output

```
Sorted array (Deterministic):
1 2 3 5 8

Sorted array (Randomized):
1 2 3 5 8
```

Assignment 4: N-Queens Problem

.

Program

```
#include <bits/stdc++.h>
using namespace std;

int COL, ROW;
int N = 4;

void printSolution(vector<vector<int>> board) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i][j]) cout << "Q ";
            else cout << ". ";
        }
        cout << endl;
    }
}

bool isSafe(vector<vector<int>> board, int row, int col) {
    int i, j;
    for (i = 0; i < col; i++)
        if (board[row][i]) return false;
    for (i = col + 1; i < N; i++)
        if (board[row][i]) return false;
    for (i = row, j = col; i < N && j < N; i++, j++)
        if (board[i][j]) return false;
    for (i = row, j = col; i >= 0 && j < N; i--, j++)
        if (board[i][j]) return false;
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j]) return false;
    for (i = row, j = col; j >= 0 && i < N; i++, j--)
        if (board[i][j]) return false;
    return true;
}

bool solveNQUtil(vector<vector<int>>& board, int col) {
    if (col >= N) return true;
    if (col == COL) return solveNQUtil(board, col + 1);

    for (int i = 0; i < N; i++) {
        if (isSafe(board, i, col)) {
            board[i][col] = 1;
            if (solveNQUtil(board, col + 1)) return true;
            board[i][col] = 0;
        }
    }
    return false;
}
```

```
bool solveNQ() {
    cout << "Enter size of board: ";
    cin >> N;
    cout << "Enter row and col of first queen to be placed:\n";
    cout << "row (1-" << N << "): ";
    cin >> ROW;
    ROW--;
    cout << "\ncol(1-" << N << "): ";
    cin >> COL;
    COL--;
    cout << endl;
    vector<vector<int>> board(N, vector<int>(N, 0));
    board[ROW][COL] = 1;
    if (!solveNQUtil(board, 0)) {
        cout << "Solution does not exist";
        return false;
    }
    printSolution(board);
    return true;
}

int main() {
    solveNQ();
    return 0;
}
```

Time Complexity (TC)

Worst case: O(N!) (backtracking search).

Space Complexity (SC)

O(N^2) for the board, plus O(N) recursion stack.

Input

```
4
1
1
```

Output

```
Enter size of board: 4
Enter row and col of first queen to be placed:
row (1-4): 1

col(1-4): 1

Q . . .
. . . Q
. Q . .
. . Q
```

Assignment 3: 0/1 Knapsack Using Dynamic Programming

Program

```
#include <bits/stdc++.h>
using namespace std;
struct Item {
    int weight;
    int value;
};

int knapsack(int W, vector<Item>& items) {
    int n = items.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (items[i - 1].weight <= w) {
                dp[i][w] = max(
                    dp[i - 1][w],
                    dp[i - 1][w - items[i - 1].weight] + items[i - 1].value
                );
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[n][W];
}

int main() {
    int W = 269;
    vector<Item> items = {
        {95, 55}, {4, 10}, {60, 47}, {32, 5},
        {23, 4}, {72, 50}, {80, 8}, {62, 61},
        {65, 85}, {46, 87}
    };
    int max_value = knapsack(W, items);
    cout << "Maximum value: " << max_value << endl;
    return 0;
}
```

Time Complexity (TC)

O(nW), where n is the number of items and W is the capacity.

Space Complexity (SC)

O(nW) for the DP table.

Output

```
Maximum value: 269
```

Assignment 2: Huffman Encoding

Program

```
#include <iostream>
#include <queue>
#include <vector>
#include <unordered_map>
using namespace std;

struct HuffmanNode {
    char data;
    int frequency;
    HuffmanNode *left, *right;

    HuffmanNode(char data, int frequency) {
        left = right = nullptr;
        this->data = data;
        this->frequency = frequency;
    }
};

struct Compare {
    bool operator()(HuffmanNode* left, HuffmanNode* right) {
        return left->frequency > right->frequency;
    }
};

void printCodes(HuffmanNode* root, string code,
                unordered_map<char, string>& huffmanCodes) {
    if (!root) return;
    if (!root->left && !root->right) {
        huffmanCodes[root->data] = code;
        cout << root->data << ": " << code << endl;
    }
    printCodes(root->left, code + "0", huffmanCodes);
    printCodes(root->right, code + "1", huffmanCodes);
}

void buildHuffmanTree(const string& text) {
    unordered_map<char, int> frequencyMap;
    for (char ch : text) frequencyMap[ch]++;

    priority_queue<HuffmanNode*, vector<HuffmanNode*>, Compare> pq;
    for (auto pair : frequencyMap)
        pq.push(new HuffmanNode(pair.first, pair.second));

    while (pq.size() > 1) {
        HuffmanNode* left = pq.top();
        pq.pop();
        HuffmanNode* right = pq.top();
```

```
        pq.pop();

        HuffmanNode* newNode =
            new HuffmanNode('\0', left->frequency + right->frequency);
        newNode->left = left;
        newNode->right = right;
        pq.push(newNode);
    }

    HuffmanNode* root = pq.top();
    unordered_map<char, string> huffmanCodes;
    printCodes(root, "", huffmanCodes);

    cout << "\nEncoded string: ";
    for (char ch : text) cout << huffmanCodes[ch];
    cout << endl;
}

int main() {
    string text;
    cout << "Enter text to encode using Huffman Encoding: ";
    getline(cin, text);
    buildHuffmanTree(text);
    return 0;
}
```

Time Complexity (TC)

O(k log k + n), where k is the number of distinct characters and n is text length.

Space Complexity (SC)

O(k) for the frequency map, priority queue, tree, and codes (excluding output).

Input

```
hello world
```

Output

```
Enter text to encode using Huffman Encoding: hello world
d: 000
r: 001
w: 010
h: 011
l: 10
o: 110
e: 1110
 : 1111

Encoded string: 011111010100110110111111110010001
```

Assignment 1: Fibonacci — Recursive and Iterative

Program

```
#include <iostream>
using namespace std;

int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

void recursive(int n) {
    for (int i = 0; i < n; ++i)
        cout << fibonacci(i) << " ";
}

void iterative(int n) {
    int a1 = 0;
    int a2 = 1;
    cout << a1 << " " << a2 << " ";
    for (int i = 2; i < n; i++) {
        int a3 = a2 + a1;
        a1 = a2;
        a2 = a3;
        cout << a2 << " ";
    }
    cout << endl;
}

int main() {
    recursive(10);
    cout << endl;
    iterative(10);
    return 0;
}
```

Time Complexity (TC)

Recursive: O(2^n) time. Iterative: O(n) time.

Space Complexity (SC)

Recursive: O(n) call stack. Iterative: O(1) auxiliary space.

Input

10

Output

```
Recursive :0 1 1 2 3 5 8 13 21 34
Iteravtive :0 1 1 2 3 5 8 13 21 34
```